EEE301 Signals and Systems

Title: Introduction to using MATLAB/OCTAVE and plotting basic functions.

Instruction: Use MATLAB/OCTAVE to complete the following laboratory exercises in Section A, B and C. You should follow the section by section and code along with the examples.

Section A: Using MATLAB command windows

1. Declare and initialize a sequence $x[n] = [5, 4, 3, 2]$ by typing the following in the MATLAB command window and press Enter:

```
>> x = [5, 4, 3, 2]
```

2. Each of the items in the sequence $x[n]$ can be accessed according to index. For example, sequence indexed 2 can be accessed by typing:

```
>> x(2)
```

Important: MATLAB index begins from 1. The first element of the MATLAB vector/array is $x(1)$.Therefore, trying to access $x(0)$ will result in error.

3. A convenient method to create a series of number starting from initial value ***init*** and ends with final value ***end*** with increment of ***inc*** is to use the following command ***init:inc:end***. For example, create the sequence $y[n] = [2, 4, 6, 8, 10]$ by typing:

```
>> y = [2:2:10]
```

4. To append new values to an existing sequence, for example, append $x[n] = [1, 2, 3, 4]$ to $y[n] = [2, 4, 6, 8, 10]$ to create $z(n) = \{2, 4, 6, 8, 10, 1, 2, 3, 4\}$, use the following command:

```
>> z = [y, x]
```

5. MATLAB provides some useful and common built-in functions for user convenience, for example mean(), max(), min(), etc.
Type the following commands in command window and write down the answers:

MATLAB commands	Answers	Descriptions of functions
$max([2, 4, 3, 1, 2])$		
$min([-0.5, 0.1, 0.35, 1.1])$		
$mean([2, 4, 6, 8])$		
$std([2, 4, 6, 8])$		
$mode([1, 3, 7, 4, 2, 1, 2, 2])$		
$median([2, 3, 1, 5, 0])$		

$sum([1,2,3,4,5])$		

6. MATLAB provides arithmetic operations such as addition ($+$), subtraction ($-$) , multiplication ($*$), division ($/$), power of ($\hat{}$) and element-wise operation ($.$)

If $x1 = [1,3,5,7]$ and $x2 = [2,4,6,8]$, find the results of the following operations using MATLAB command windows:

MATLAB operations	Answers
`x1 + x2`	
`x1 - x2`	
`2*x1`	
`x2/2`	
`x1*x2'`	
`x1.*x2`	
`x1.^2`	

Note: If $x = [1\ 2\ 3]$, then MATLAB expression x' produces $[1;\ 2;\ 3]$ or the matrix transpose of x.
And also x. ^2 means every element of matrix x is multiplied by 2. This is called element-wise product. Similarly, x1.$*$ x2 means element-wise multiplication between corresponding elements in matrix x1 and x2.

7. MATLAB provides logical operations and comparison operations such as AND (&), OR (|), NOT ($\sim$), more than ($>$), less than ($<$), more than or equal to ($>=$), less than or equal to ($<=$), equal to ($==$), not equal to ($\sim =$).
If $i = 5$ and $j = 6$, find the results of the following operations using MATLAB command windows:

MATLAB operations	Answers
$i == 5$	
$i > j$	
$i >= j-1$	
$(j < 7)\ \&\ (i > 5)$	
$(j < 7)\ |\ (i > 5)$	
$j \sim = (i + 1)$	

8. The *if*, *elseif* and *else* statements can be written using the syntax:

If <condition1>
    <command1>
elseif <condition2>
    <command2>
else
    <command3>
end

As an example, run the following code in MATLAB command window:

```
>> i = 1;
>> if (i == 1)
      disp('i is 1.');
   else
      disp('i is not 1.');
   end
```

Type the command above in MATLAB command window and observe the output.

9. A *for* loop can be written as shown in example below.

```
>> for i = 1:1:5
      sprintf('i is %d.',i)
   end
```

Type the command above in MATLAB command window and observe the output.

10. The function `linspace(x1, x2, n)` is used to generate evenly spaced n number of points between two endpoints x1 and x2. This function is useful to generate the time variable t. For example, to generate the variable time t defined between $-5 \leq t \leq 5$, use the command below:

```
>> t = linspace(-5, 5, 11)
```

You should obtain `t = [-5 -4 -3 -2 -1 0 1 2 3 4 5]`.
Alternatively, you can always use

```
>> t = [-5:1:5]
```

Section B: Using m-file

1. A more convenient way to use MATLAB is to save all code in M-file which has the extension **.m**. To do this, click "New Script" on the "Home" tab. A new window will pop out to be used as code editor. The empty space is where the code is to be written.

2. As an example, type the following code in the MATLAB m-file editor.

```
function [average, stddev] = statsresults(x)
% compute mean and standard deviation of x
average = mean(x);
stddev = std(x);
```

Save the file above as statsresults.m in a current folder.
Next, execute the m-file above by typing the following at the command window.

```
>> [ave, stddev] = statsresults([0.2, 0.4, 0.6, 0.8, 1.0])
```

What are the outputs of the function? `ave` = _____, `stddev` = _____.

3. Write your own m-file with function

`[x1, t1] = time_inverse(x, t)`

that will take a sequence of samples of signal $x(t)$ and produce the time inverse version of the signal, that is, $x(-t)$. For example, if $x(t) = [0, 0, 0, 2, 4, 6]$ for time $t = [-2, -1, 0, 1, 2, 3]$, then the function `time_inverse(x, t)` will produce $x1\ (t1) = [6\ 4\ 2\ 0\ 0\ 0]$ for $t1 = [-3\ -2\ -1\ 0\ 1\ 2]$.

4. Repeat question 3 above for function

`[x1, t1] = time_shift(x, t, t0)`

that will take a sequence of samples of signal $x(t)$ and produce the time shifted version of the signal by t_0, that is, $x(t-t0)$. For example, if $x(t) = [0\ 0\ 0\ 2\ 4\ 6]$ for time $t = [-2\ -1\ 0\ 1\ 2\ 3]$, then the function `time_shift(x, t, 2)` will produce $x1(t1) = [0\ 0\ 0\ 2\ 4\ 6]$ for $t1 = [0\ 1\ 2\ 3\ 4\ 5]$.

Note: Plot the input and output function `time_inverse` and `time_shift` for question B.3 and B.4 as:

```
>> figure,
>> plot(t, x); hold on;
>> plot(t1, x1)
```

Section C: Plotting Continuous-Time and Discrete-time Signals using MATLAB.

1. Continuous time signals can be plotted using the MATLAB function `plot()`. An example is shown below. Copy and save code below into M-file. Run and observe the output plot.

```
t = 0:0.01:2; % sample points from 0 to 2 in steps of 0.01
x = sin(2*pi*t); % Evaluate sin(2 pi t)
plot(t,x,'b'); % Create plot with blue line
xlabel('t in sec'); ylabel('x(t)'); % Label axis
title('Plot of sin(2\pi t)'); % Title plot
```

2. Discrete time signals can plotted using the MATLAB function stem(). Run the code below in MATLAB and observe the output plot.

```
n = 0:1:40; % sample index from 0 to 20
x = sin(0.1*pi*n); % Evaluate sin(0.1 pi n)
Hs = stem(n,x,'b','filled'); % Stem-plot with handle Hs
set(Hs,'markersize',4); % Change circle size
xlabel('n'); ylabel('x(n)'); % Label axis
title('Stem Plot of sin(0.1\pi n)'); % Title plot
```

3. We can combine the two plots above into a single plot using the function subplot(). Run the code below in MATLAB and observe the output plot.

```
t = 0:0.01:2; % sample points from 0 to 2 in steps of 0.01
x = sin(2*pi*t); % Evaluate sin(2 pi t)


subplot(2,1,1); % Two rows, one column, first plot


plot(t,x,'b'); % Create plot with blue line
xlabel('t in sec'); ylabel('x(t)'); % Label axis
title('Plot of sin(2\pi t)'); % Title plot


n = 0:1:40; % sample index from 0 to 20
x = sin(0.1*pi*n); % Evaluate sin(0.2 pi n)


subplot(2,1,2); % Two rows, one column, second plot


Hs = stem(n,x,'b','filled'); % Stem-plot with handle Hs
xlabel('n'); ylabel('x(n)'); % Label axis
title('Stem Plot of sin(0.2 pi n)'); % Title plot
```

4. Next, try to write your own MATLAB code and save it in an M-file for the following questions.

Following functions can be expanded as a sum of even and odd function.

$$x(t) = x_e(t) + x_o(t),$$

where,

$$x_e = \frac{1}{2}[x(t) + x(-t)]$$

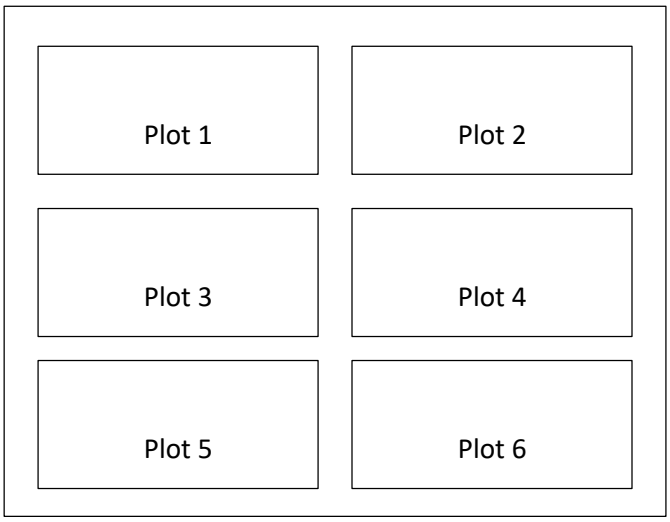
$$x_o(t) = \frac{1}{2}[x(t) - x(-t)].$$

Complete following exercise:

a) $x_1(t) = e^{-2t}\,u(t)$ for $-2 \le t \le 2$ on plot 1.
b) the even signal of $x_1(t)$, i.e. $x_{1e}(t)$ for $-2 \le t \le 2$ on plot 3,
c) the odd signal of $x_1(t)$, i.e. $x_{1o}(t)$ for $-2 \le t \le 2$ on plot 5,

d) $x_2(t) = -0.5[u(t+1) - u(t)] + t[u(t) - u(t-2)]$ for $-2 \le t \le 2$ on plot 2,
e) the even signal of $x_2(t)$, i.e. $x_{2e}(t)$ for $-2 \le t \le 2$ on plot 4,
f) the odd signal of $x_2(t)$, i.e. $x_{2o}(t)$for $-2 \le t \le 2$ on plot 6.

Refer to the plot arrangement below.



Label your plots accordingly. [**Hint**: You can explore the MATLAB function heaviside(t) and fliplr(x) for this question. Refer to Appendix]

Appendix

MATLAB functions

MATLAB functions	Description
`heaviside(t)`	Returns the value 0 for $t < 0$ or the value 1 for $t > 0$ or the value ½ for $t = 0$.
`fliplr(x)`	Returns x with its columns flipped in the left-right direction.
`roots(p)`	Returns the roots of polynomial represented by p as a column vector. Input p is a vector containing n+1 polynomial coefficients starting with the coefficients of x^n.